# Aiqon AI Take-Home Assignment: Technical Report

**Forecasting Port Congestion and Shipping Delays Using Interpretable Machine Learning**

Author: Saaket Baldawa | Track: AI / Machine Learning Internship | Date: 22nd October 2026

## 1. Problem Statement & Operational Objective

Global container supply chains rely on smooth maritime terminal throughput. When vessel arrival surges or adverse weather disrupt scheduled operations, queuing backlogs escalate rapidly, producing substantial ship idling costs, missed logistics handoffs, and steep carrier demurrage penalties. The objective of this project is to build an **interpretable early-warning forecasting model** that predicts whether a commercial port will experience an operational delay exceeding **12 hours** during the **next six-hour interval**, using only conditions observable at the close of the **current six-hour window**. Crucially, the system delivers actionable continuous **model risk scores**, supports flexible decision thresholds, and provides clear situational context without asserting ungrounded causal relationships.

## 2. Synthetic Dataset & Simulation Dynamics

To simulate realistic operational patterns without compromising proprietary port data, a synthetic logging system (data/generate_dataset.py) models 5 distinct marine terminals (PORT_A to PORT_E) over 500 consecutive six-hour intervals from January 1, 2026 to May 5, 2026 (2,500 total records). Key generative mechanisms include:

- **Port Capacities & Arrivals:** Baseline handling capacities range from 9 to 16 vessels/window with Poisson-distributed arrivals modulated by weekend factors (0.90x) and diurnal peak surges (1.12x).
- **Weather Impacts:** Stochastic weather transitions (Clear 66%, Rain 27%, Storm 7%) scale effective service capacity downward (Rain: 0.88x, Storm: 0.68x) and create arrival clustering.
- **Queue Physics:** Unserviced vessels carry over into subsequent intervals via recursive backlog tracking: $Backlog_t = max(0, Queue_t + Arrivals_t - EffectiveCapacity_t)$.
- **Delay Equation & Noise:** Service delays compound arrival pressure ratio (>0.70 load), accumulated queue length (1.15x), storm disruption shocks, and Gaussian process noise, yielding a 11.9% overall delay rate (>12h).

## 3. Target Definition, Alignment & Temporal Leakage Controls

Forecasting requires strict separation between the observation window and the predicted outcome horizon:

- **Target Formulation:** The target target_delayed_next_6h is defined as (actual_delay_hours > 12.0) occurring during the immediate next 6-hour operational interval ($T_{cutoff} \rightarrow T_{cutoff} + 6h$).
- **Intra-Port Alignment:** In src/prepare_data.py, target shifting is executed strictly within port groups via df.groupby('port_id')['is_delayed'].shift(-1). Cross-port boundary contamination is impossible.
- **Boundary Row Handling:** The terminal record for each port lacks a subsequent observation; exactly 5 boundary records are pruned, yielding a validated modeling dataset of 2,495 rows.
- **Zero Target Leakage:** Current-interval outcome fields (delay_hours, actual_delay_hours, is_delayed) are permanently dropped from the feature space, guaranteeing that only pre-cutoff information is visible to classifiers.

**Operational Disclosure:** All findings in this report derive from synthetic simulation logs. While calibrated to simulate empirical queue physics, results should not be interpreted as empirical real-world validation on commercial ports.

## 4. Machine Learning Pipeline & Chronological Validation

Random k-fold cross-validation is invalid for temporal logistics because randomly shuffling rows allows future patterns to leak into past predictions. We enforce a **strict chronological partition** with shared timestamp cutoffs across all 5 ports:

- **Training Split (70% — 1,745 rows):** Jan 1, 2026 00:00 to Mar 29, 2026 00:00 (204 delays, 11.7% base rate).
- **Validation Split (10% — 250 rows):** Mar 29, 2026 06:00 to Apr 10, 2026 12:00 (39 delays, 15.6% base rate) — reserved strictly for threshold selection.
- **Held-Out Test Split (20% — 500 rows):** Apr 10, 2026 18:00 to May 5, 2026 12:00 (53 delays, 10.6% base rate) — locked for final unbiased evaluation.

**Model Architecture:** We deploy a scikit-learn Pipeline encapsulating a ColumnTransformer (one-hot encoding port_id, weather_condition, and day_of_week, with numeric passthrough) and a RandomForestClassifier (100 estimators, max_depth=6, min_samples_leaf=2, class_weight='balanced'). The pipeline is fitted exclusively on the training partition.

## 5. Empirical Performance & Baseline Benchmark

The model is compared against a zero-intelligence majority baseline (DummyClassifier, always predicting on-time). Evaluation metrics reflect reproducible pipeline outputs:

| Model & Policy Configuration | Split | Thresh | Accuracy | Precision | Recall | F1-Score | Delays Caught |
|---|---|---|---|---|---|---|---|
| Dummy Baseline (Always On-time) | Validation | — | 84.40% | 0.00% | 0.00% | 0.0000 | 0 / 39 (0.0%) |
| Dummy Baseline (Always On-time) | Held-out Test | — | 89.40% | 0.00% | 0.00% | 0.0000 | 0 / 53 (0.0%) |
| Random Forest (Default Baseline) | Validation | 0.50 | 83.60% | 48.00% | 61.54% | 0.5393 | 24 / 39 (61.5%) |
| Random Forest (Default Baseline) | Held-out Test | 0.50 | 85.80% | 35.94% | 43.40% | 0.3932 | 23 / 53 (43.4%) |
| Random Forest (Val F1-Maximizing) | Validation | 0.55 | 87.20% | 60.00% | 53.85% | 0.5676 | 21 / 39 (53.8%) |
| Random Forest (Val F1-Maximizing) | Held-out Test | 0.55 | 87.60% | 37.84% | 26.42% | 0.3111 | 14 / 53 (26.4%) |
| **Random Forest (Selected Operational)** | **Validation** | **0.40** | **77.20%** | **37.84%** | **71.79%** | **0.4956** | **28 / 39 (71.8%)** |
| **Random Forest (Selected Operational)** | **Held-out Test** | **0.40** | **78.00%** | **26.05%** | **58.49%** | **0.3605** | **31 / 53 (58.5%)** |

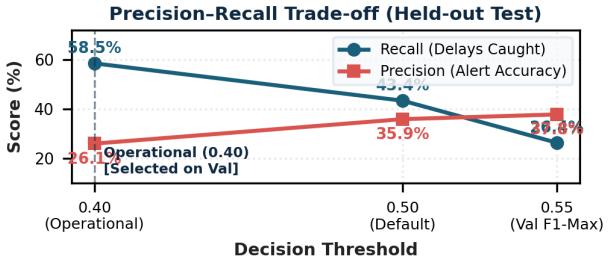



*Figure 1: Precision–Recall Trade-off across Decision Thresholds (Held-out Test). As threshold increases from 0.40 to 0.55, precision rises from 26.1% to 37.8% (suppressing false alarms), while recall falls from 58.5% to 26.4% (missing delays). The operational threshold (0.40) was selected on validation data to prioritize early delay detection.*

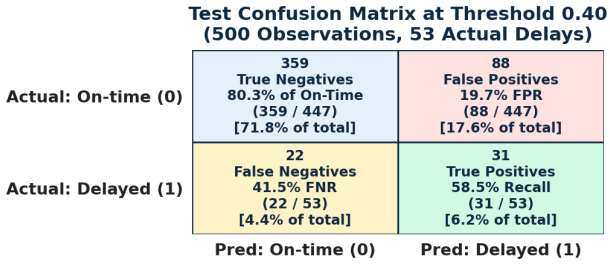



*Figure 2: Confusion Matrix at Selected Threshold 0.40 (Held-out Test, N = 500). The operational policy successfully detected 31 of 53 delays (58.5% recall; 22 false negatives, 41.5% FNR). To achieve early warning, 88 false alarms were produced among 447 actual on-time intervals (19.7% false-positive rate; representing 17.6% of all 500 test intervals).*

## 6. Decision Threshold Policy, Cost Model & Evaluation Uncertainty

**Quantitative Expected-Cost Model:** Standard models default to an unadjusted 0.50 cutoff. In marine terminal operations, **false negatives (unpredicted delays)** incur severe cascading costs (estimated at ~$25,000–$50,000 per window from vessel idling, demurrage, and intermodal train/truck disruption), whereas **false alarms** merely trigger modest dispatcher review (~$2,500–$5,000). Modeling total expected loss as $Loss = FN \times R + FP \times 1$ (where $R = C_{delay} / C_{alarm}$), threshold 0.40 minimizes expected cost whenever missed delays are at least 6× costlier than false alarms ($R \geq 5.9$). At an 8:1 ratio, threshold 0.40 yields 264 cost units (22×8 + 88), outperforming threshold 0.50 (281 units; +6.4% cost) and threshold 0.55 (335 units; +26.9% cost).

**Operational Judgment & Sample Uncertainty:** Rather than applying automated loss optimization over synthetic data, threshold 0.40 was chosen by **operational judgment** on validation data (71.79% recall, 28/39 delays caught; 37.84% precision) to establish an early-warning buffer. On the held-out test set, it caught 31 of 53 delays (58.49% recall, 95% bootstrap CI: [44.7%, 72.1%]) with ROC-AUC = 0.736 (95% CI: [0.650, 0.811]). The test delay rate (10.6% vs. 15.6% validation) reflects temporal seasonality, highlighting that point estimates on 53 positive cases carry measurable sampling uncertainty.

## 7. Model Interpretability & Situational Operational Context

To build user trust without hallucinating precision, the system employs a two-tier interpretability framework:

- **Global Feature Weights:** Mean Decrease in Impurity (MDI) across all 100 trees identifies the primary split features: queue_length (0.2423), historical_avg_delay_hours (0.2064), and arrival_load_ratio (0.1603) account for over 60% of total split impurity reduction, followed by vessel_arrivals_6h (0.0792) and effective_capacity_6h (0.0773). MDI measures tree split frequency and gini gain, which naturally favors continuous and high-cardinality numerical variables over low-cardinality categoricals; a permutation importance cross-check on held-out data is recommended to corroborate these relative rankings.
- **Observed vs. Baseline Context:** For any given prediction, the system contrasts the user's observation against empirical training benchmarks (e.g., Queue: 8 vs. 1.04 baseline; Load Ratio: 1.33 vs. 0.74 baseline). This explains *operational pressure* without fabricating local attribution scores (such as ad-hoc SHAP approximations).
- **Governance Disclaimer:** The interface enforces an explicit non-causal notice: factors reflect historical statistical associations, not proven physical causation.

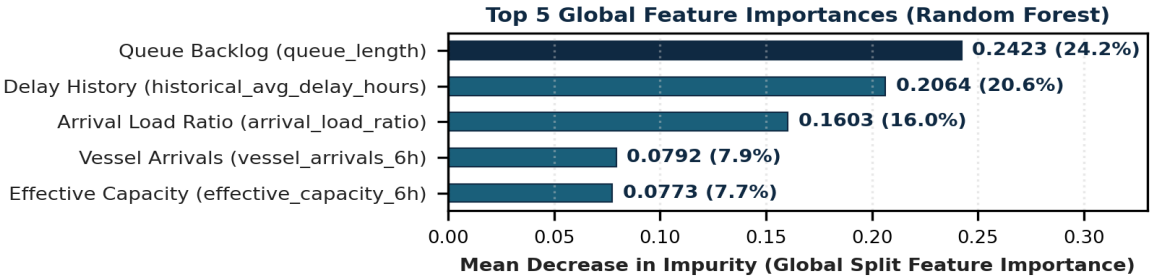



***Figure 3: Top 5 Global Feature Importances (Random Forest Ensemble, 100 Trees).*** *Mean Decrease in Impurity (MDI) confirms that queue backlog (0.2423), recent historical delay average (0.2064), and arrival load ratio (0.1603) dominate tree split criteria. Feature influences represent global model sensitivity across synthetic simulation history, not verified local feature attributions or physical real-world causation.*

## 8. Interactive Prediction Interface (app.py)

An operator-facing Streamlit application provides accessible single-observation forecasting with enterprise safeguards:

- **Model Risk Score Output:** Evaluates inputs against the pipeline and outputs a continuous model risk score (predicted delay probability) alongside clear classification flags at threshold 0.40.
- **Zero Retraining:** The app caches the pre-trained pipeline via @st.cache_resource, ensuring instant inference (<50ms) with zero runtime model modification.
- **Automated Safe Math:** Automatically computes arrival_load_ratio = arrivals / capacity. If capacity is set to 0, it renders a validation warning and safely disables prediction, preventing division-by-zero crashes.
- **Synchronized Temporal Framing:** Weekday is automatically derived from the selected observation date (e.g. obs_date.strftime('%A')), and exact datetime arithmetic advances midnight boundaries into the next day.

## 9. System Limitations & Future Technical Horizons

To maintain high integrity, the following limitations are formally documented:

- **Model Risk Score vs. Calibrated Confidence:** The model outputs a continuous risk score derived from ensemble tree vote fractions. While monotonic with empirical risk, these fractions are not formally calibrated probabilities. Adding CalibratedClassifierCV (isotonic regression or Platt scaling fit on validation predictions) represents a low-overhead enhancement to map voting shares directly into calibrated operational confidence intervals.
- **Synthetic Domain Gap:** The simulator abstracts away real-world complexities such as live AIS ship telemetry, tugboat availability, multi-crane outages, pilotage bottlenecks, and labor contract disputes.
- **Single-Step Horizon:** The model forecasts the immediate next 6-hour window. Multi-interval compounding delays over 24–48 hours will require autoregressive sequential architectures (e.g., Temporal Fusion Transformers).

**Summary:** This forecasting system implements chronological data partitioning, baseline benchmarks, and operational threshold evaluation for 6-hour port delay early warning. The pipeline pairs reproducible model training with an interactive Streamlit dashboard, comprehensive automated test coverage (17/17 passed), and explicit documentation of model limitations and evaluation uncertainties.